



CCFR3: A cooperative co-evolution with efficient resource allocation for large-scale global optimization

Ming Yang^{a,b}, Aimin Zhou^c, Xiaofen Lu^{b,*}, Zhihua Cai^a, Changhe Li^d, Jing Guan^e

^a School of Computer Science, China University of Geosciences, No. 388 Lumo Road, Wuhan, 430074, Hubei, China

^b Guangdong Provincial Key Laboratory of Brain-inspired Intelligent Computation, Department of Computer Science and Engineering, Southern University of Science and Technology, 1088 Xueyuan Avenue, Shenzhen, 518055, Guangdong, China

^c School of Computer Science and Technology, East China Normal University, 500 Dongchuan Road, Shanghai, 200062, China

^d School of Automation, China University of Geosciences, No. 388 Lumo Road, Wuhan, 430074, Hubei, China

^e China Ship Development and Design Center, No. 268 Zhangzhidong Road, Wuhan, 430064, Hubei, China

ARTICLE INFO

Keywords:

Cooperative co-evolution
Large-scale global optimization
Decomposition
Resource allocation

ABSTRACT

Cooperative co-evolution (CC) adopts the divide-and-conquer strategy to decompose an optimization problem, which can decrease the difficulty of solving large-scale optimization problems. Each decomposed subproblem is solved by a subpopulation. According to the contributions of the subpopulations to the improvement of the best overall objective value, the CC algorithms select the subpopulation with the greatest contribution to undergo evolution. In the existing CC algorithms, the contribution evaluation cannot adapt to solve the optimization problem, which may decrease the performance of CC. In this paper, we propose a new CC framework named CCFR3, which can adaptively evaluate the contribution of a subpopulation in each co-evolutionary cycle. CCFR3 can allocate computational resources among subpopulations more frequently than other contribution-based CC algorithms. The subpopulations can have more chances to undergo evolution, which is beneficial to speed up the convergence of CC and enhance the performance of CC on obtaining the global optimal solution. Our experimental results and analysis suggest that CCFR3 is a competitive solver for large-scale optimization problems.

1. Introduction

Optimization, which has been lecture and laboratory parts of the education syllabus (Precup et al., 2021), widely exists in the real world, where people aim to obtain the optimal solution to the problem to be solved, such as in predicting the factors in people's behavioral intention (Tan, Ooi, Leong, & Lin, 2014), enhancing the position tracking performance of electromagnetic actuated clutch systems (Bojan-Dragos et al., 2021), and the optimal controls in nonlinear servo systems (Precup, David, Roman, Szedlak-Stinean and Petriu, 2021) and the medical field (Rigatos, Siano, Selisteanu, & Precup, 2017). Large-scale optimization problems involve more than a thousand of decision variables. Due to the huge search space and number of local optima of large-scale optimization problems, the performance of evolutionary algorithms (EAs) (Sarker, Mohammadian, & Yao, 2002) may decrease greatly on solving this kind of problem.

Cooperative co-evolution (CC) (Potter & Jong, 1994) adopts divide-and-conquer strategy (Blanchard, Beauthier, & Carletti, 2019; Peng, Jin, & Wang, 2019; Peng, Liu, & Jin, 2016; Yang, Tang, & Yao, 2019)

to solve large-scale optimization problems. CC decomposes a problem into several subproblems. Each subproblem involves a subcomponent of decision variables of the original problem. Correspondingly, CC divides the population into several subpopulations. Each subpopulation is responsible for optimizing a subcomponent of variables. By selecting each subpopulation to undergo evolution for GEs generations, where GEs is a prespecified parameter with a fixed value, CC solves the subproblems separately.

During the cooperative co-evolution, different subpopulations are very likely to make different contributions to the improvement of the best overall objective value (Yang et al., 2017). More computational resources should be allocated to the subpopulation with greater contribution. According to the contributions of subpopulations, the contribution-based CC algorithms (Omidvar, Kazimipour, Li, & Yao, 2016; Omidvar, Li, & Yao, 2011; Yang et al., 2017; Yang, Zhou, Li, Guan, & Yan, 2020) allocate computational resources among the subpopulations. For a subpopulation which is selected to undergo

* Corresponding author.

E-mail addresses: mingyang@cug.edu.cn (M. Yang), amzhou@cs.ecnu.edu.cn (A. Zhou), luxf@sustech.edu.cn (X. Lu), zhcai@cug.edu.cn (Z. Cai), lichanghe@cug.edu.cn (C. Li), whsm3998d@whzyh.cn (J. Guan).

<https://doi.org/10.1016/j.eswa.2022.117397>

Received 3 November 2021; Received in revised form 10 February 2022; Accepted 25 April 2022

Available online 6 May 2022

0957-4174/© 2022 Elsevier Ltd. All rights reserved.

evolution for GEs generations, the contribution-based CC algorithms compute the improvement of the best overall objective value as the contribution of the subpopulation. The existing contribution-based CC cannot adaptively determine when to compute the contribution of a subpopulation.

The performance of contribution-based CC is affected by the value of GEs (see Section 2.2). The improvement of the best overall objective value decreases as the evolution of a subpopulation progresses. If GEs is set to a large value, the improvement is not significant in the late evolutionary period, e.g., the subpopulation is converging at an optimum. The computational resources should be saved for the evolution of another subpopulation. If the value of GEs is too small, it may not be suitable to evaluate the contribution of a subpopulation according to the limited evolutionary information. An inaccurate contribution evaluation may result in an inefficient resource allocation among subpopulations.

In this paper, we propose an improved version of contribution based CC which removes parameter GEs . According to the information on the contribution to improvement of the best overall objective value, the improved CC can adaptively determine whether to interrupt the evolution of a subpopulation and select another subpopulation to undergo evolution, which can make CC better respond to the improvement of the best overall objective value.

The remainder of this paper is organized as follows. Section 2 presents an overview of CC. Section 3 introduces our new CC. Section 4 presents the experimental studies. Finally, Section 5 concludes the paper.

2. CC frameworks

In this section, we review the original CC and the contribution-based CC. An experimental study is presented to show the effect of GEs on the performance of contribution-based CC.

2.1. Original CC

In the original cooperative co-evolutionary genetic algorithm (CCGA) (Potter & Jong, 1994), a D -dimensional problem is decomposed into D one-dimensional subproblems. This decomposition strategy is not suitable to solve the nonseparable problems because it ignores the interrelationship between decision variables (Weise, Chiong, & Tang, 2012).

Several decomposition methods (Shi, Teng, & Li, 2005; Van den Bergh & Engelbrecht, 2004) randomly group the decision variables into several subcomponents of variables. The interrelated variables may be grouped into one subcomponent with a probability. However, the decomposition is fixed during the running of CC. To increase the probability of grouping the interrelated variables together, the decomposition is dynamic during the running of CC in Li and Yao (2009, 2012), Omidvar, Li, Yang, and Yao (2010) and Yang, Tang, and Yao (2008a, 2008b) through randomly grouping the decision variables in each co-evolutionary cycle. If the effect of the variation of a variable on the function value is independent on the effect of the variation of another variable on the function value, the two variables are separable (Tezuka, Munetomo, & Akama, 2004). The differential grouping methods (Hu, He, Chen, & Zhang, 2017; Mei, Omidvar, Li, & Yao, 2016; Omidvar, Li, Mei, & Yao, 2014; Omidvar, Yang, Mei, Li, & Yao, 2017; Sun, Kirley, & Halgamuge, 2015; Sun, Kirley, & Halgamuge, 2018; Sun, Omidvar, Kirley, & Li, 2018; Yang, Zhou, Li, & Yao, 2021) based on this theory can identify the interrelationship between variables with a high accuracy.

Suppose $C = \{C_1, \dots, C_M\}$ is a decomposition of variables, where C_i is a subcomponent of variables, and $\bar{P} = \{\bar{x}_1, \dots, \bar{x}_N\}$ is a population. The subpopulation for optimizing C_i is $\bar{P}_i$, where $\bar{P}_i$ consists of the elements of $\bar{P}$ in the dimensions of C_i . $\bar{x}_{best}$ is the combination of the best known individual from each subpopulation. For subpopulation $\bar{P}_i$,

Algorithm 1 CCGA (Potter & Jong, 1994)

```

1: for  $k \leftarrow 1$  to  $Cy$  do
2:   for  $i \leftarrow 1$  to  $M$  do
3:     for  $GEs$  generations do
4:        $\bar{P}'_i \leftarrow \{(\bar{x}, \bar{x}_{best}^{\hat{P}_i}) | \bar{x} \in \bar{P}_i\}$ ;
5:       Evaluate  $\bar{P}'_i$ ;
6:       Apply genetic operators to  $\bar{P}'_i$ ;
7:        $\bar{P}''_i \leftarrow \{(\bar{x}, \bar{x}_{best}^{\hat{P}_i}) | \bar{x} \in \bar{P}'_i\}$ ;
8:       Evaluate  $\bar{P}''_i$ ;
9:        $\bar{P}_i \leftarrow$  Selection between  $\bar{P}'_i$  and  $\bar{P}''_i$ ;
10:      Update  $\bar{x}_{best}$ ;
11:   end for
12: end for
13: end for

```

an overall solution combines an individual from $\bar{P}_i$ and $\bar{x}_{best}^{\hat{P}_i}$, where $\bar{x}_{best}^{\hat{P}_i}$ is $\bar{x}_{best}$ without the dimensions from $\bar{P}_i$. Algorithm 1 illustrates the CCGA framework. One co-evolutionary cycle is comprised of Steps 2 to 12. In CCGA, $GEs = 1$ (see Step 3), i.e., subpopulations undergo only one evolutionary generation in each co-evolutionary cycle. When a decomposition is generated, each subproblem is solved by CCGA using an evolutionary optimizer in a round-robin fashion. CCGA allocates equal computational resources to solve the subproblems. However, the subproblems are most likely to have different effect on the best overall objective value. The computational resource allocation in CCGA is not reasonable to solve this kind of problem.

2.2. CCFR

CCFR (Yang et al., 2017), a contribution-based CC, adopts a two-stage approach to solving an optimization problem. In the first stage, CCFR decomposes the problem into several subproblems by a grouping method. In the second stage, using an optimizer, CCFR solves the subproblems separately.

According to the contributions of subpopulations to the improvement of the best overall objective value, CCFR allocates computational resources to subpopulations. In each co-evolutionary cycle, unlike the original CC where all the subpopulations are selected to undergo evolution, CCFR selects only the subpopulation with the greatest contribution to undergo evolution. The contribution of a subpopulation is computed as follows.

$$Z_i = \frac{\hat{Z}_i + |f(\bar{x}_{best}^{\hat{P}_i}) - f(\bar{x}_{best})|}{2}, \quad (1)$$

where $\bar{x}_{best}^{\hat{P}_i}$ and $\bar{x}_{best}$ are the best overall solution before and after $\bar{P}_i$ undergoes evolution in a co-evolutionary cycle respectively, f is the objective function of the solved optimization problem, and $\hat{Z}_i$ is the last contribution of $\bar{P}_i$. The initial value of $\hat{Z}_i$ is zero. Considering the historical and real-time contribution information, CCFR adapts the contribution of a subpopulation as the co-evolution progresses by Eq. (1). CCFR adopts the method in Yang, Li, Cai, and Guan (2015) to identify stagnant subpopulations and stops the evolution of the stagnant subpopulation, which can save computational resources. The experimental results in Yang et al. (2017) showed that CCFR can make better use of computational resources than CCGA and is a highly competitive CC framework for solving large-scale optimization problems.

CCFR specifies equal-sized subpopulations for optimizing the variables of different subproblems. However, different subproblems may have different characteristics. A small-sized subpopulation is sufficient to solve an easy subproblem, which can speed up the convergence of the algorithm. To solve a difficult subproblem, an optimization

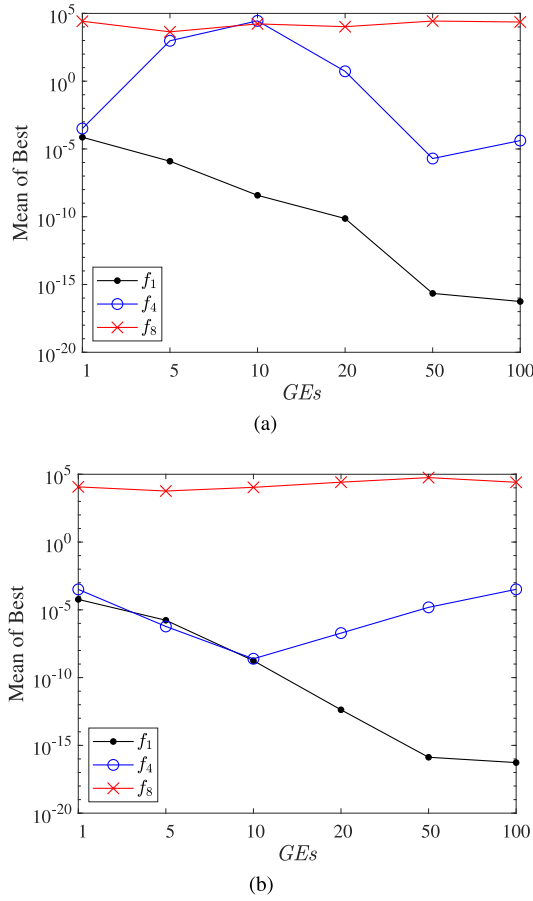


Fig. 1. Average fitness obtained by CC algorithms with different values of GEs over 25 independent runs on selected CEC'2013 functions f_1 , f_4 and f_8 . (a) CCFR. (b) CCFR2.

algorithm usually needs a large-sized subpopulation, which can enhance the global search capacity of the algorithm. According to the dimensionality of a subproblem, CCFR2 (Yang et al., 2020) specifies unequal-sized subpopulations for solving different subproblems. The experimental results in Yang et al. (2020) showed that this simple idea can significantly improve the performance of CCFR on solving the large-scale optimization problems.

CCFR and CCFR2 select a subpopulation to undergo evolution for GEs generations in each co-evolutionary cycle. The contribution of a subpopulation is updated every GEs generations. The value of GEs is prespecified and fixed. Different values of GEs result in obtaining different contributions of subpopulations, which may affect the computational resource allocation. Fig. 1 shows the average fitness value obtained by CCFR and CCFR2 with different values of GEs over 25 independent runs on the CEC'2013 functions f_1 , f_4 and f_8 . f_1 is a fully separable function. f_4 is a partially separable function with seven subcomponents of nonseparable variables and 700 separable variables. f_8 is a partially separable function with 20 subcomponents of nonseparable variables. It can be seen in Fig. 1 that the performance of CCFR and CCFR2 is affected by the value of GEs. There does not exist a suitable value of GEs for CCFR and CCFR2 to solve f_1 , f_4 and f_8 at the same time. The value of GEs should be adaptive to solve different problems.

3. New contribution-based CC

An improved CCFR2 named CCFR3 is presented in this section. According to the historical improvements of the best overall objective value, CCFR3 can adaptively determine whether to stop the evolution

of a subpopulation and update the contribution of a subpopulation. The parameter GEs is removed in CCFR3.

3.1. Self-adaptive contribution computation

For a subpopulation ($\vec{P}_i$), suppose $\vec{x}'_{best}$ is the best overall solution at the beginning of the evolution of $\vec{P}_i$. During the evolution of $\vec{P}_i$, when the real-time best overall solution $\vec{x}_{best}$ is different from $\vec{x}'_{best}$ in all the dimensions of C_i , the evolution of $\vec{P}_i$ is suspended. In the above evolution interval (since the beginning of the evolution until the evolution of $\vec{P}_i$ is suspended), the improvement of the best overall objective value is computed as follows.

$$\Delta F_1 = \frac{|f(\vec{x}'_{best}) - f(\vec{x}_{best})|}{n_1}, \quad (2)$$

where n_1 is the number of fitness evaluations $\vec{P}_i$ spends in the evolution interval. ΔF_1 is the improvement of the best overall objective value per one fitness evaluation. $\vec{P}_i$ starts a new evolution interval. Similarly, we can obtain ΔF_2 according to Eq. (2).

Repeat the above process until $s_1 = 1$, where s_1 is a flag to denote whether to stop the evolution of $\vec{P}_i$. At each evolutionary generation, the value of s_1 is set as follows.

$$s_1 = \begin{cases} 1 & \text{if } j > 2 \text{ and } std_j < std_{j-1} \\ 0 & \text{otherwise,} \end{cases} \quad \begin{matrix} (a) \\ (b) \end{matrix} \quad (3)$$

where std_{j-1} and std_j are the standard deviations of $\{\Delta F_1, \dots, \Delta F_{j-1}\}$ and $\{\Delta F_1, \dots, \Delta F_j\}$ respectively. $\Delta F_1, \dots, \Delta F_j$ are the improvements of the best overall objective value in the evolution intervals [see Eq. (2)]. If the standard deviation of the improvements becomes smaller, we deem that $\vec{P}_i$ makes a stable contribution to the improvement of the best overall objective value and set s_1 to 1. When $s_1 = 1$, we stop the evolution of $\vec{P}_i$. The contribution of $\vec{P}_i$ to the improvement of the best overall objective value is computed as follows.

$$Z_i = \frac{|f(\vec{x}''_{best}) - f(\vec{x}_{best})|}{n}, \quad (4)$$

where $\vec{x}''_{best}$ and $\vec{x}_{best}$ are the best overall solution before and after $\vec{P}_i$ undergoes evolution respectively and n is the total number of fitness evaluations $\vec{P}_i$ spends in the above evolution intervals. Because $\vec{P}_i$ makes a stable contribution to the improvement of the best overall objective value, we can set the real-time improvement of the best overall objective value in one co-evolutionary cycle as the contribution of $\vec{P}_i$.

If $\vec{x}_{best}$ remains unchanged, e.g., $\vec{P}_i$ is trapped in an optimum, s_1 is always equal to 0. $\vec{P}_i$ would keep undergoing evolution until the algorithm meets the termination criterion. It will waste the computational resources that $\vec{P}_i$ continues the evolution. We stop the evolution of $\vec{P}_i$ when $s_2 = 1$, where s_2 is a flag to denote whether the evolution of $\vec{P}_i$ is stopped. At each evolutionary generation, the value of s_2 is set as follows.

$$s_2 = \begin{cases} 1 & \text{if } U > N_i \text{ and } d \leq d' \\ 0 & \text{otherwise.} \end{cases} \quad \begin{matrix} (a) \\ (b) \end{matrix} \quad (5)$$

U is the number of the successive generations where $\vec{x}_{best}$ remains unchanged and N_i is the population size of $\vec{P}_i$. For a given optimization problem, the larger the population size is, the more generations the population would spend converging to an optimum. Therefore, we set the population size of $\vec{P}_i$ as the threshold value of U . d' and d are the sum of the distances of the individuals in $\vec{P}_i$ to $\vec{x}_{best}^{P_i}$ before and after U generations respectively, where $\vec{x}_{best}^{P_i}$ is the individual in $\vec{P}_i$ with the best fitness value. If $\vec{x}_{best}$ remains unchanged for U successive generations and $d < d'$, $\vec{P}_i$ might be converging to an optimum. We can stop the evolution of $\vec{P}_i$ early but not until the whole population is convergent at the optimum. If $\vec{x}_{best}$ remains unchanged for U successive generations and $d = d'$, the evolution of $\vec{P}_i$ might be stagnant. To save

Algorithm 2 CCFR3

```

1: Generate a decomposition  $C = \{C_1, \dots, C_M\}$  using a variable grouping
   method;
2: Generate uniform random subpopulations  $\bar{P}_i, i = 1, 2, \dots, M$ ;
3: Set  $\bar{x}_{best}$  to a combination of a randomly chosen individual from each
   subpopulation;
4:  $Z_i \leftarrow 0, i = 1, 2, \dots, M$ ;
5: Set  $\bar{x}'_{P_i, best}, i = 1, 2, \dots, M$ , to empty vectors;
6: while the termination criterion is not met do
7:   for  $i \leftarrow 1$  to  $M$  do
8:      $(\bar{P}_i, \bar{x}_{best}, Z_i) \leftarrow \text{Optimizer}(\bar{x}_{best}, \bar{P}_i, \bar{x}'_{P_i, best}, C_i)$ ;
9:      $\bar{x}'_{P_i, best} \leftarrow \bar{x}_{best}$ ;
10:   end for
11:   while  $\min(Z_i | i = 1, \dots, M) \neq \max(Z_i | i = 1, \dots, M)$  do
12:      $i \leftarrow$  the index of the maximum  $Z_i$ ;
13:      $(\bar{P}_i, \bar{x}_{best}, Z_i) \leftarrow \text{Optimizer}(\bar{x}_{best}, \bar{P}_i, \bar{x}'_{P_i, best}, C_i)$ ;
14:      $\bar{x}'_{P_i, best} \leftarrow \bar{x}_{best}$ ;
15:   end while
16: end while

```

computational resources, the evolution of $\bar{P}_i$ should also be stopped. In the case that the evolution of $\bar{P}_i$ is stopped and $\bar{P}_i$ does not make a stable contribution to the improvement of the best overall objective value (i.e., $s_1 = 0$ and $s_2 = 1$), the contribution of $\bar{P}_i$ is set as follows.

$$Z_i = \begin{cases} 0.5 \cdot \hat{Z}_i + 0.5 \cdot \frac{|f(\bar{x}''_{best}) - f(\bar{x}_{best})|}{n} & \text{if } d < d' \quad (a) \\ 0 & \text{otherwise,} \quad (b) \end{cases} \quad (6)$$

where $\hat{Z}_i$ is the last contribution of $\bar{P}_i$. In the case that $\bar{P}_i$ converges to an optimum, the contribution of $\bar{P}_i$ is set to a statistical value (the average of the historical and real-time contributions of $\bar{P}_i$), which is suggested in Yang et al. (2017). The more recent improvement of the best objective value is, the greater the effect of the improvement on the contribution computation is. The effects of the early improvements of the best objective value become smaller and smaller as the co-evolution progresses. In the case that the evolution of $\bar{P}_i$ is stagnant, the contribution of $\bar{P}_i$ is set to zero [see Eq. (6)(b)].

If $\bar{P}_i$ makes a stable contribution to the improvement of the best overall objective value, $\bar{P}_i$ stops evolution and the algorithm reselects a subpopulation to undergo evolution. Compared with the existing strategy that $\bar{P}_i$ undergoes evolution for GEs generations, the above strategy can stop the evolution of $\bar{P}_i$ early and improve the frequency of the computational resource allocation among subpopulations, which can speed up the convergence of the algorithm. If $\bar{P}_i$ is convergent or stagnant, the contribution of $\bar{P}_i$ is set to 0 because the best objective value cannot be improved any more. The subpopulations whose contributions are equal to zero will not be selected to undergo evolution, i.e., the convergent and stagnant subpopulations are excluded from the evolution. When all the subpopulations are convergent or stagnant, the contributions of all the subpopulations are equal to zero.

3.2. CCFR3

Similar to CCFR2 (Yang et al., 2020), CCFR3 selects the subpopulation with the greatest contribution to the improvement of the best overall objective value to undergo evolution in each co-evolutionary cycle. The experimental study in Yang et al. (2017, 2020) showed that the contribution-driven strategy can significantly improve the performance of CC on solving large-scale optimization problems. Algorithm 2 illustrates the framework of CCFR3, which is the same with CCFR2. Steps 11 to 15 select the subpopulation with the greatest contribution to undergo evolution. When all the subpopulations make the same contribution, CCFR3 moves to Step 7. All the subpopulations are selected to undergo evolution one by one. A subpopulation employs an optimizer

Algorithm 3 $(\bar{P}_i, \bar{x}_{best}, Z_i) \leftarrow \text{Optimizer}(\bar{x}_{best}, \bar{P}_i, \bar{x}'_{P_i, best}, C_i)$

```

1: for  $\bar{x} \in \bar{P}_i$  do
2:   if  $\bar{x}_{best} \neq \bar{x}'_{P_i, best}$  then
3:     Evaluate  $(\bar{x}, \bar{x}_{best})$  and update the number of fitness evaluations (i.e.,
       num);
4:   end if
5: end for
6:  $\bar{x}'_{best} \leftarrow \bar{x}_{best}$ ;  $\bar{x}''_{best} \leftarrow \bar{x}_{best}$ ;  $\bar{x}'''_{best} \leftarrow \bar{x}_{best}$ ;  $num' \leftarrow num$ ;  $S_{\Delta F} \leftarrow \emptyset$ ;  $std' \leftarrow 0$ ;
    $U \leftarrow 0$ ;
7: while true do
8:   for  $\bar{x} \in \bar{P}_i$  do
9:      $\bar{x}' \leftarrow \text{Reproduction}(\bar{x})$ ; /*evolutionary process*/
10:    Evaluate  $(\bar{x}', \bar{x}_{best})$  and update num;
11:    if  $(\bar{x}', \bar{x}_{best})$  is better than  $(\bar{x}, \bar{x}_{best})$  then
12:       $\bar{x} \leftarrow \bar{x}'$ ;
13:    end if
14:    if  $(\bar{x}', \bar{x}_{best})$  is better than  $\bar{x}_{best}$  then
15:       $\bar{x}_{best} \leftarrow (\bar{x}', \bar{x}_{best})$ ;
16:    end if
17:  end for
18:   $s_1 \leftarrow 0$ ;
19:  if  $\bar{x}'_{best}$  is different from  $\bar{x}_{best}$  in all the dimensions of  $C_i$  then
20:     $\Delta F \leftarrow (|f(\bar{x}'_{best}) - f(\bar{x}_{best})|) / (num - num')$ ;
21:     $S_{\Delta F} \leftarrow S_{\Delta F} \cup \{\Delta F\}$ ;  $std \leftarrow$  the standard deviations of  $S_{\Delta F}$ ;
22:    if  $|S_{\Delta F}| > 2$  and  $std < std'$  then
23:       $s_1 \leftarrow 1$ ;
24:    end if
25:     $\bar{x}'_{best} \leftarrow \bar{x}_{best}$ ;  $num' \leftarrow num$ ;  $std' \leftarrow std$ ;
26:  end if
27:   $s_2 \leftarrow 0$ ;
28:  if  $\bar{x}'''_{best}$  equals to  $\bar{x}_{best}$  in all the dimensions of  $C_i$  then
29:     $d \leftarrow \sum_{\bar{x} \in \bar{P}_i} \|\bar{x} - \bar{x}_{best}\|$ ;
30:    if  $U = 0$  then
31:       $d' \leftarrow d$ ;
32:    end if
33:    if  $d \leq d'$  then
34:       $U \leftarrow U + 1$ ;
35:      if  $U > N_i$  then /* $N_i$  is the population size of  $P_i$ */
36:         $s_2 \leftarrow 1$ ;
37:      end if
38:    end if
39:  else
40:     $U \leftarrow 0$ ;
41:  end if
42:   $\bar{x}'''_{best} \leftarrow \bar{x}_{best}$ ;
43:  if  $s_1 = 1$  then
44:    Set  $Z_i$  according to Eq. (4), return;
45:  else if  $s_2 = 1$  then
46:    Set  $Z_i$  according to Eq. (6), return;
47:  end if
48: end while

```

(i.e., Algorithm 3) to take the evolutionary process (see Steps 8 and 13).

Algorithm 3 illustrates the evolution of a subpopulation ($\bar{P}_i$). $\bar{P}_i$ evolves until $\bar{P}_i$ makes a stable contribution to the improvement of the best overall objective value or $\bar{P}_i$ is convergent or stagnant (see Steps 7 to 48). Steps 18 to 26 check whether $\bar{P}_i$ makes a stable contribution. If $\bar{P}_i$ makes a stable contribution [i.e., $s_1 = 1$, see Eq. (3)], Step 44 evaluates the contribution of $\bar{P}_i$ as the real-time improvement of the best overall objective value [see Eq. (4)]. Steps 27 to 41 check whether $\bar{P}_i$ is convergent or stagnant. If $\bar{P}_i$ is convergent or stagnant [i.e., $s_2 = 1$, see Eq. (5)], Step 46 evaluates the contribution of $\bar{P}_i$ as the average of the historical and real-time contributions of $\bar{P}_i$ [see Eq. (6)], which is used in CCFR (Yang et al., 2017). CCFR3 executes Steps 18 to 47 to determine when to stop the evolution of $\bar{P}_i$ and compute the contribution of $\bar{P}_i$, where CCFR3 is different from CCFR2. The time

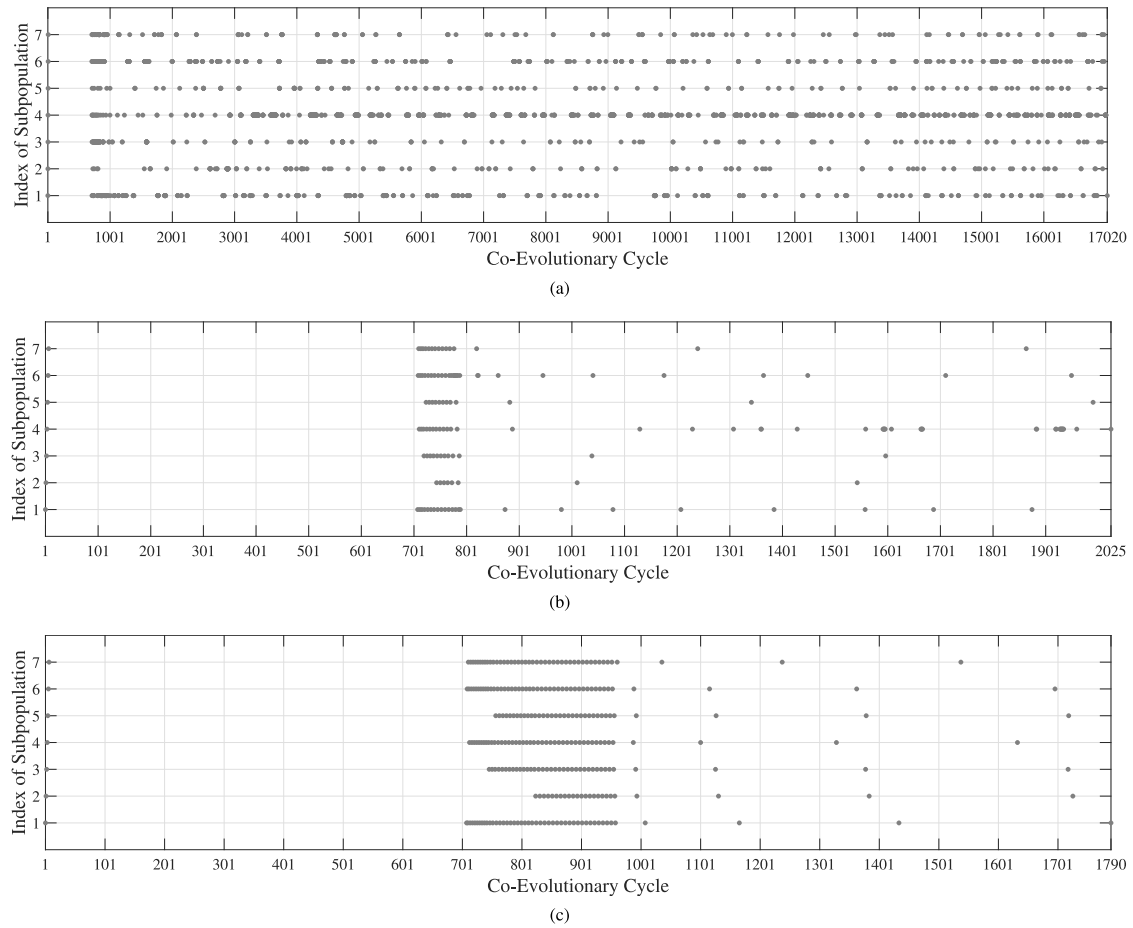


Fig. 2. For the same initial population, activation of subpopulations on f_7 before 10^6 fitness evaluations in a single run, where the filled circle point indicates that the subpopulation undergoes evolution in the corresponding co-evolutionary cycle. (a) CCFR3. (b) CCFR2. (c) CCFR.

Table 1

Grouping results of ERDG on the CEC'2013 functions: used fitness evaluations (FEs), number of decomposed groups and grouping accuracy.

F	Used FEs	# of groups		Accuracy	
		Sep	Non-Sep	Sep	Non-Sep
f_1	2998	1000	0	100.0%	–
f_2	2998	1000	0	100.0%	–
f_3	3996	0	1	0.0%	–
f_4	5326	700	7	100.0%	100.0%
f_5	5395	700	7	100.0%	100.0%
f_6	5905	0	7	0.0%	91.7%
f_7	5554	700	7	100.0%	100.0%
f_8	8451	200	17	–	75.0%
f_9	8812	0	20	–	100.0%
f_{10}	8794	0	20	–	87.5%
f_{11}	9212	0	20	–	100.0%
f_{12}	26 980	0	1	–	100.0%
f_{13}	7599	0	2	–	0.0%
f_{14}	8420	0	1	–	100.0%
f_{15}	3996	0	1	–	100.0%

complexity of Steps 18 to 47 is $\mathcal{O}(D_i)$, where D_i is the dimensionality of C_i .

4. Experimental studies

The performance of CCFR3 is studied in this section. Section 4.1 presents the effect of the improvement proposed in Section 3.1 on the contribution-based CC. Section 4.2 presents the comparison between

CCFR3 and its competitors. A set of 15 1000-dimensional test instances proposed in the IEEE CEC'2013 special session on large-scale global optimization were used in the experimental studies. The detailed description of the CEC'2013 test instances is given in Li, Tang, Omidvar, Yang, and Qin (2013). The CEC'2013 test instances are classified into the following five categories.

1. Fully separable functions (f_1 – f_3).
2. Partially separable functions with 700 separable variables and seven subcomponents of nonseparable variables (f_4 – f_7).
3. Partially separable functions with 20 subcomponents of nonseparable variables (f_8 – f_{11}).
4. Nonseparable functions with overlapping subcomponents (f_{12} – f_{14}).
5. Fully nonseparable function (f_{15}).

We set the maximum number of fitness evaluations to 3×10^6 as the termination criterion, which is suggested in Li et al. (2013).

CCFR3 is compared with two contribution-based CC algorithms [CCFR (Yang et al., 2017) and CCFR2 (Yang et al., 2020)] and the original CC (Potter & Jong, 1994). All the CC algorithms adopted CMA-ES (Hansen, 2005) as the optimizer, which is recommended in CCFR and CCFR2. The parameters of CCFR, CCFR2 and the original CC were set as in their publications, e.g., the parameter of GEs was set to 100 as in CCFR and CCFR2. The CC algorithms used an efficient recursive differential grouping (ERDG) method (Yang et al., 2021) to decompose the optimization problems. Table 1 summarizes the grouping results of ERDG on the CEC'2013 functions. Compared with other grouping methods, ERDG can correctly group the interrelated variables together with much fewer fitness evaluations for almost all the IEEE CEC'2013

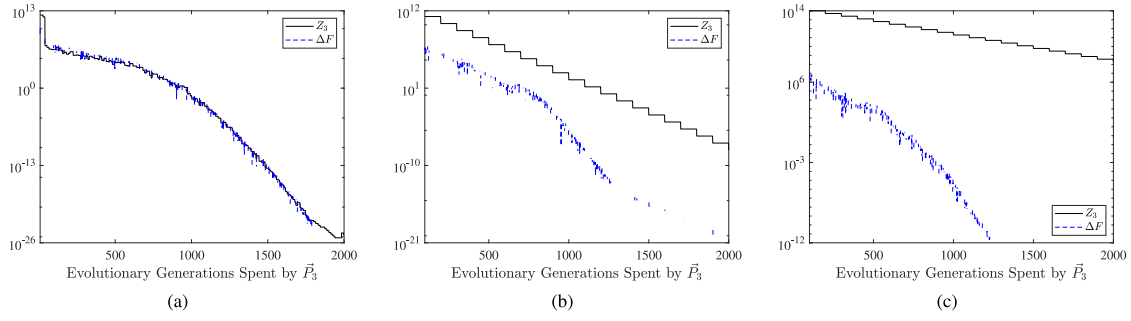


Fig. 3. For the same initial population, the contribution of $\bar{P}_3$ on f_7 in a single run. Because the value in axis Y is the logarithm, the line may be broken. (a) CCFR3. (b) CCFR2. (c) CCFR.

Table 2

For the same initial population, the numbers of co-evolutionary cycles (Cy_i , $i = 1, \dots, M$) the M subpopulations undergo in a single run on the CEC'2013 fully and partially separable functions.

FEs	F	CCFR3				CCFR2				CCFR				CC			
		min Cy_i	max Cy_i	$\bar{C}y_i$	$\sum_{i=1}^M Cy_i$	min Cy_i	max Cy_i	$\bar{C}y_i$	$\sum_{i=1}^M Cy_i$	min Cy_i	max Cy_i	$\bar{C}y_i$	$\sum_{i=1}^M Cy_i$	min Cy_i	max Cy_i	$\bar{C}y_i$	$\sum_{i=1}^M Cy_i$
1e6	f_1	39	84	48.7	48 665	1	6	2.8	2824	1	7	2.9	2869	2	3	2.5	2493
	f_2	12	58	36.8	36 812	1	5	2.6	2596	1	7	2.7	2660	2	3	2.5	2493
	f_3	1493	1493	1493	1493	416	416	416	416	416	416	416	416	416	416	416	416
	f_4	5	2490	21.2	14 975	1	31	3.2	2289	1	35	3.2	2249	3	4	3.4	2416
	f_5	23	173	43.8	30 954	1	11	3.5	2478	1	26	3.4	2408	3	4	3.4	2416
	f_6	182	1181	507.1	3550	43	208	91.1	638	56	178	89.9	629	91	92	91.1	638
	f_7	9	1560	24	16 938	1	45	3	2102	1	47	2.5	1785	3	4	3.4	2416
	f_8	2	1657	31	6727	1	171	3.8	825	1	157	3.7	798	9	10	9.4	2037
	f_9	104	316	173.3	3465	23	105	33.3	666	20	55	33.9	678	34	35	34.2	684
	f_{10}	24	479	98.3	1965	6	196	32.9	657	1	98	34.3	685	34	35	34.6	693
	f_{11}	10	819	206.3	4127	8	127	31.3	626	13	77	33.5	670	34	35	34.2	684
3e6	f_1	164	209	173.7	173 665	443	449	444.5	444 488	441	448	442.7	442 696	7	8	7.5	7493
	f_2	118	203	144.8	144 768	362	371	365.9	365 879	364	373	368.1	368 066	7	8	7.5	7493
	f_3	4029	4029	4029	4029	1249	1249	1249	1249	1249	1249	1249	1249	1249	1249	1249	1249
	f_4	16	4380	43.9	31 064	2	342	6.7	4723	2	328	6.6	4686	10	11	10.3	7292
	f_5	35	366	62.5	44 214	1	180	6.6	4697	1	191	6.7	4730	10	11	10.3	7292
	f_6	385	3114	1045.1	7316	105	713	232.7	1629	182	468	255	1785	274	275	274.7	1923
	f_7	16	2553	35	24 756	3	302	6.7	4750	3	270	6.6	4679	10	11	10.3	7291
	f_8	11	3114	213.8	46 388	1	255	14.4	3133	1	244	15.4	3338	28	29	28.6	6196
	f_9	197	410	266.4	5328	90	205	102	2040	89	124	102.8	2057	103	104	103.2	2063
	f_{10}	77	1752	474.1	9481	18	324	95.8	1917	18	322	97.8	1955	104	105	104.5	2091
	f_{11}	160	2085	674.5	13 491	29	287	93.7	1873	58	200	99	1980	103	104	103.2	2063

functions (Yang et al., 2021). The fitness evaluations spent by ERDG are counted as part of the computational budget.

4.1. Study on resource allocation

f_7 is decomposed by ERDG into 707 subcomponents of variables including seven subcomponents of nonseparable variables and 700 subcomponents of separable variables, where ERDG decomposes f_7 with the accuracy of 100% (see Table 1). CCFR3 significantly outperforms its competitors on f_7 [see Table 3 and Fig. 5(g)]. For the same initial population, Fig. 2 shows the activation of subpopulations on f_7 before 10^6 fitness evaluations in a single run. The seven subcomponents of nonseparable variables make great effect on the overall objective value. Fig. 2 shows the subpopulations which are responsible for optimizing the seven subcomponents of nonseparable variables. Therefore, there are no circle pints at some co-evolutionary cycle points where separable variables undergoes evolution in these co-evolutionary cycles. At the point of 10^6 fitness evaluations, there were 17 020 co-evolutionary cycles in CCFR3, while there were 2025 and 1790 co-evolutionary cycles in CCFR2 and CCFR respectively. CCFR3 allocated computational resources more frequently than CCFR2 and CCFR. It can be seen in Fig. 2 that the seven subpopulations in CCFR3 had more chances to undergo evolution than the ones in CCFR2 and CCFR. This was also true for the other subpopulations which are responsible for optimizing the 700 separable variables. CCFR3 spent 13 895 co-evolutionary cycles

optimizing the separable variables, while CCFR2 and CCFR spent 1968 and 1512 co-evolutionary cycles respectively. This is because a subpopulation in CCFR2 and CCFR spends a fixed number of evolutionary generations (i.e., 100 evolutionary generations) in each co-evolutionary cycle, while CCFR3 adaptively determines the number of evolutionary generations (may be smaller than 100) according to the evolutionary information. Under a fixed computational budget, all the subpopulations in CCFR3 can have more chances to undergo evolution than CCFR2 and CCFR. Fig. 3 shows the contributions of $\bar{P}_3$ computed by CCFR3, CCFR2 and CCFR on f_7 in a single run. Z_3 is the contribution of $\bar{P}_3$ computed by an algorithm. ΔF is the improvement of the best overall objective value per fitness evaluation over each evolutionary generation. It can be seen in Fig. 3 that Z_3 computed by CCFR3 [see Eq. (4)] is consistent with the real improvement of the best overall objective value, while Z_3 computed by CCFR2 and CCFR [see Eq. (1)] is not consistent with improvement of the best overall objective value. The contribution evaluations in CCFR2 and CCFR consider the historical improvement of the best overall objective value, which results in that the contribution evaluations in CCFR2 and CCFR respond worse to the real improvement of the best overall objective value than the contribution evaluation in CCFR3. The more accurate contribution evaluation in CCFR3 can make better use of computational resources. Therefore, CCFR3 outperforms CCFR2 and CCFR by several orders of magnitude on f_7 , where CCFR3 obtains the global optimal solution at the point of 10^6 fitness evaluations but CCFR2 and CCFR not (see Table 3).

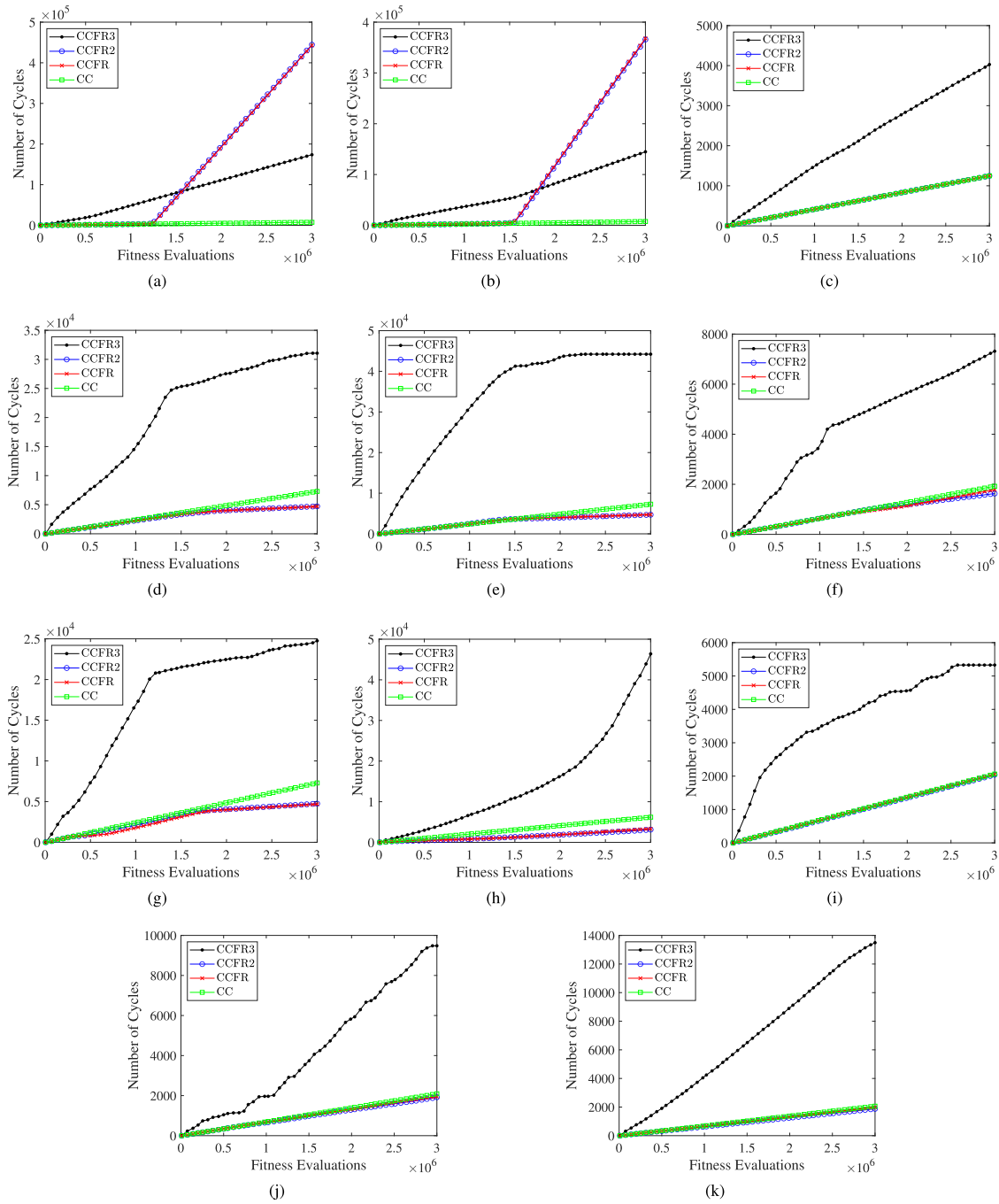


Fig. 4. For the same initial population, the numbers of co-evolutionary cycles during a single run of CC algorithms on the CEC'2013 fully and partially separable functions. (a) f_1 . (b) f_2 . (c) f_3 . (d) f_4 . (e) f_5 . (f) f_6 . (g) f_7 . (h) f_8 . (i) f_9 . (j) f_{10} . (k) f_{11} .

Fig. 4 shows the numbers of co-evolutionary cycles during a single run of CC algorithms on the CEC'2013 fully and partially separable functions f_1 – f_{11} . The numbers of co-evolutionary cycles in CCFR3 were significantly larger than the ones in CCFR2 and CCFR on almost all the 11 functions. CCFR3 can allocate computational resources among subpopulations more frequently than CCFR2 and CCFR during the running of an algorithm. CCFR3 determines the evolution of the subpopulation is stagnant if the distribution of the subpopulation remains unchanged in five successive generations, while CCFR2 and CCFR determine a subpopulation is stagnant if the distribution of the subpopulation remains unchanged in two successive generations. Therefore, for the fully separable functions f_1 and f_2 , the numbers of co-evolutionary cycles in CCFR3 were smaller than the ones in CCFR2 and CCFR in the late stage of the running of the algorithm [see Figs. 4(a) and 4(b)].

For the fully and partially separable functions f_1 – f_{11} , we counted the number of co-evolutionary cycles each subpopulation undergoes. Table 2 summarizes the numbers of co-evolutionary cycles in a single run. The total numbers of co-evolutionary cycles (i.e., $\sum_{i=1}^M Cy_i$) in CCFR3 were larger than the ones in CCFR2 and CCFR on almost all the functions not only at the end of the running of the algorithms but also in the middle of the running. This indicated that all the subpopulations in CCFR3 had more chances to undergo evolution than the ones in CCFR2 and CCFR on almost of the functions f_1 – f_{11} .

4.2. Comparison on optimization

Table 3 summarizes the optimization results of the CC algorithms on the CEC'2013 functions over 25 independent runs. There is no great

Table 3

Average fitness values $\pm$ standard deviations over 25 independent runs on the CEC'2013 functions. The significantly best results are in bold font (Wilcoxon test with Holm p -value correction, $\alpha = 0.05$). b , n , and l denote the number of functions where CCFR3 performs significantly better than, statistically equivalent to, and significantly worse than its competitors, respectively.

FES	F	CCFR3	CCFR2	CCFR	CC
1e6	f_1	1.10e-09 $\pm$ 2.40e-09	1.16e-03 $\pm$ 2.00e-04 $\uparrow$	2.15e-03 $\pm$ 2.77e-04 $\uparrow$	4.58e-17 $\pm$ 3.96e-18
	f_2	4.37e+02 $\pm$ 3.12e+01	4.32e+02 $\pm$ 3.23e+01	4.32e+02 $\pm$ 3.23e+01	4.32e+02 $\pm$ 3.23e+01
	f_3	2.06e+01 $\pm$ 3.02e-02	2.06e+01 $\pm$ 2.98e-02	2.06e+01 $\pm$ 3.23e-02	2.06e+01 $\pm$ 3.21e-02
	f_4	1.29e-03 $\pm$ 2.71e-03	2.18e+06 $\pm$ 5.70e+05 $\uparrow$	2.23e+06 $\pm$ 6.84e+05 $\uparrow$	2.47e+10 $\pm$ 1.23e+10 $\uparrow$
	f_5	2.46e+06 $\pm$ 4.89e+05	2.39e+06 $\pm$ 4.82e+05	2.39e+06 $\pm$ 5.69e+05	2.44e+06 $\pm$ 5.00e+05
	f_6	9.98e+05 $\pm$ 1.36e+04	9.99e+05 $\pm$ 1.35e+04	9.99e+05 $\pm$ 1.31e+04	1.00e+06 $\pm$ 4.43e+03 $\uparrow$
	f_7	1.57e-13 $\pm$ 7.25e-13	4.48e+03 $\pm$ 2.04e+03 $\uparrow$	7.66e+03 $\pm$ 5.09e+03 $\uparrow$	1.84e+08 $\pm$ 1.05e+08 $\uparrow$
	f_8	3.32e+05 $\pm$ 9.22e+04	6.71e+05 $\pm$ 1.22e+05 $\uparrow$	1.31e+06 $\pm$ 2.05e+05 $\uparrow$	3.84e+13 $\pm$ 2.84e+13 $\uparrow$
	f_9	1.67e+08 $\pm$ 2.98e+07	1.54e+08 $\pm$ 2.91e+07	1.55e+08 $\pm$ 2.84e+07	1.61e+08 $\pm$ 3.24e+07
	f_{10}	9.37e+07 $\pm$ 1.83e+06	9.37e+07 $\pm$ 1.73e+06	9.18e+07 $\pm$ 1.70e+06$\downarrow$	9.38e+07 $\pm$ 1.46e+06
	f_{11}	8.56e+01 $\pm$ 3.83e+01	3.18e+02 $\pm$ 1.71e+02 $\uparrow$	4.37e+05 $\pm$ 1.48e+05 $\uparrow$	5.38e+07 $\pm$ 2.14e+07 $\uparrow$
	f_{12}	1.08e+03 $\pm$ 6.79e+01	1.08e+03 $\pm$ 6.79e+01	1.08e+03 $\pm$ 6.79e+01	1.08e+03 $\pm$ 6.79e+01
	f_{13}	2.50e+07 $\pm$ 3.23e+07	1.48e+07 $\pm$ 8.32e+06	1.19e+07 $\pm$ 2.25e+06 $\downarrow$	2.02e+07 $\pm$ 3.56e+06 $\downarrow$
	f_{14}	1.50e+08 $\pm$ 1.68e+07	1.50e+08 $\pm$ 1.68e+07	1.50e+08 $\pm$ 1.68e+07	1.50e+08 $\pm$ 1.68e+07
	f_{15}	8.89e+06 $\pm$ 9.09e+05	8.89e+06 $\pm$ 9.07e+05	8.89e+06 $\pm$ 9.07e+05	8.89e+06 $\pm$ 9.07e+05
$b/n/l$		-	5/10/0	5/8/2	5/9/1
3e6	f_1	1.10e-09 $\pm$ 2.40e-09	5.26e-17 $\pm$ 6.06e-18	5.58e-17 $\pm$ 4.55e-18	1.64e-18 $\pm$ 5.04e-19
	f_2	4.37e+02 $\pm$ 3.12e+01	4.32e+02 $\pm$ 3.23e+01	4.32e+02 $\pm$ 3.23e+01	4.32e+02 $\pm$ 3.23e+01
	f_3	2.04e+01 $\pm$ 4.80e-02	2.04e+01 $\pm$ 4.79e-02	2.04e+01 $\pm$ 5.03e-02	2.04e+01 $\pm$ 5.76e-02
	f_4	1.04e-08 $\pm$ 2.76e-08	3.61e-04 $\pm$ 1.00e-04 $\uparrow$	3.73e-05 $\pm$ 3.96e-05 $\uparrow$	3.12e+08 $\pm$ 1.79e+08 $\uparrow$
	f_5	2.46e+06 $\pm$ 4.89e+05	2.39e+06 $\pm$ 4.82e+05	2.39e+06 $\pm$ 5.69e+05	2.42e+06 $\pm$ 4.95e+05
	f_6	9.95e+05 $\pm$ 1.73e+03	9.96e+05 $\pm$ 5.48e+01	9.96e+05 $\pm$ 2.19e-01 $\uparrow$	9.96e+05 $\pm$ 1.35e+02
	f_7	1.55e-13 $\pm$ 7.25e-13	2.79e-07 $\pm$ 1.19e-07 $\uparrow$	1.32e-08 $\pm$ 2.27e-08 $\uparrow$	1.03e+07 $\pm$ 5.48e+06 $\uparrow$
	f_8	4.98e+03 $\pm$ 1.27e+03	2.56e+04 $\pm$ 7.72e+03 $\uparrow$	2.62e+04 $\pm$ 1.06e+04 $\uparrow$	1.08e+09 $\pm$ 2.60e+08 $\uparrow$
	f_9	1.67e+08 $\pm$ 2.98e+07	1.54e+08 $\pm$ 2.91e+07	1.55e+08 $\pm$ 2.84e+07	1.61e+08 $\pm$ 3.24e+07
	f_{10}	9.12e+07 $\pm$ 1.31e+06	9.17e+07 $\pm$ 1.77e+06	9.07e+07 $\pm$ 2.03e+05	9.09e+07 $\pm$ 4.45e+05
	f_{11}	5.25e-18 $\pm$ 3.81e-18	5.25e-18 $\pm$ 2.81e-18	7.62e-09 $\pm$ 5.34e-09 $\uparrow$	4.03e+03 $\pm$ 5.07e+03 $\uparrow$
	f_{12}	9.90e+02 $\pm$ 6.18e+01	9.90e+02 $\pm$ 6.18e+01	9.90e+02 $\pm$ 6.18e+01	9.90e+02 $\pm$ 6.18e+01
	f_{13}	4.73e+05 $\pm$ 7.44e+04	4.92e+05 $\pm$ 7.13e+04	4.67e+05 $\pm$ 5.11e+04	8.47e+05 $\pm$ 1.44e+05 $\uparrow$
	f_{14}	2.62e+07 $\pm$ 2.03e+06	2.62e+07 $\pm$ 2.02e+06	2.62e+07 $\pm$ 2.03e+06	2.62e+07 $\pm$ 2.03e+06
	f_{15}	2.16e+06 $\pm$ 2.13e+05	2.17e+06 $\pm$ 2.17e+05	2.16e+06 $\pm$ 2.14e+05	2.16e+06 $\pm$ 2.14e+05
$b/n/l$		-	3/12/0	5/10/0	5/10/0

The symbols $\uparrow$ and $\downarrow$ denote that the CCFR3 performs significantly better than and worse than this algorithm by the Wilcoxon test at the significance level of 0.05, respectively.

Table 4

Average fitness values $\pm$ standard deviations over 25 independent runs on selected CEC'2013 functions. The significantly best results are in bold font (Wilcoxon test with Holm p -value correction, $\alpha = 0.05$). b , n , and l have similar meanings as in Table 3.

F	CCFR3	CCFR2 ($GEs = 1$)	CCFR2 ($GEs = 10$)	CCFR2 ($GEs = 50$)	CCFR2 ($GEs = 100$)
f_1	1.10e-09 $\pm$ 2.40e-09	5.87e-05 $\pm$ 2.54e-05 $\uparrow$	3.43e-09 $\pm$ 5.21e-09 $\uparrow$	8.06e-17 $\pm$ 1.24e-16	5.26e-17 $\pm$ 6.06e-18
f_2	4.37e+02 $\pm$ 3.12e+01	4.34e+02 $\pm$ 3.24e+01	4.45e+02 $\pm$ 3.53e+01	4.40e+02 $\pm$ 3.09e+01	4.32e+02 $\pm$ 3.23e+01
f_3	2.04e+01 $\pm$ 4.80e-02	2.04e+01 $\pm$ 4.79e-02	2.04e+01 $\pm$ 4.79e-02	2.04e+01 $\pm$ 4.79e-02	2.04e+01 $\pm$ 4.79e-02
f_4	1.04e-08 $\pm$ 2.76e-08	2.94e-02 $\pm$ 9.53e-02 $\uparrow$	2.32e-08 $\pm$ 9.47e-08	1.40e-05 $\pm$ 3.53e-06 $\uparrow$	3.61e-04 $\pm$ 1.00e-04 $\uparrow$
f_5	2.46e+06 $\pm$ 4.89e+05	2.29e+06 $\pm$ 3.51e+05	2.43e+06 $\pm$ 3.85e+05	2.29e+06 $\pm$ 3.44e+05	2.39e+06 $\pm$ 4.82e+05
f_6	9.95e+05 $\pm$ 1.73e+03	9.96e+05 $\pm$ 2.16e+03	9.96e+05 $\pm$ 2.53e-01	9.99e+05 $\pm$ 1.33e+04	9.96e+05 $\pm$ 5.48e+01
f_7	1.55e-13 $\pm$ 7.25e-13	9.88e-03 $\pm$ 4.81e-02 $\uparrow$	5.11e-10 $\pm$ 1.06e-09 $\uparrow$	9.56e-10 $\pm$ 2.15e-10 $\uparrow$	2.79e-07 $\pm$ 1.19e-07 $\uparrow$
f_8	4.98e+03 $\pm$ 1.27e+03	1.18e+04 $\pm$ 1.64e+04 $\uparrow$	1.75e+04 $\pm$ 2.81e+04 $\uparrow$	3.33e+04 $\pm$ 5.66e+04 $\uparrow$	2.56e+04 $\pm$ 7.72e+03 $\uparrow$
f_9	1.67e+08 $\pm$ 2.98e+07	1.61e+08 $\pm$ 2.54e+07	1.71e+08 $\pm$ 3.05e+07	1.62e+08 $\pm$ 3.67e+07	1.54e+08 $\pm$ 2.91e+07
f_{10}	9.12e+07 $\pm$ 1.31e+06	9.19e+07 $\pm$ 1.57e+06 $\uparrow$	9.21e+07 $\pm$ 1.74e+06 $\uparrow$	9.16e+07 $\pm$ 1.54e+06	9.17e+07 $\pm$ 1.78e+06
f_{11}	5.25e-18 $\pm$ 3.81e-18	5.93e-06 $\pm$ 2.22e-05 $\uparrow$	5.97e-18 $\pm$ 3.50e-18	4.89e-18 $\pm$ 3.44e-18	5.20e-18 $\pm$ 2.80e-18
$b/n/l$	-	6/5/0	4/7/0	3/8/0	3/8/0

The symbols $\uparrow$ and $\downarrow$ have similar meanings as in Table 3.

difference between the CC algorithms on f_1 – f_3 . For partially separable functions f_4 – f_{11} , CCFR3 significantly outperforms other CC algorithms on f_4 , f_7 and f_8 where the results of CCFR3 are significantly better than the ones of other CC algorithms by several orders of magnitude. At the point of 10^6 fitness evaluations, CCFR3 obtains the near global optimal solutions on f_4 and f_7 and outperforms other CC algorithms on f_4 , f_7 , f_8 and f_{11} by several orders of magnitude. CCFR3 converges faster than other CC algorithms on f_4 – f_{11} . For fully nonseparable functions, all the decision variables are grouped and optimized together. The number of evolutionary generations in each co-evolutionary cycle does not affect

the performance of CC algorithms on the fully nonseparable functions. There is no great significant difference between CCFR, CCFR2 and CBCC3 on f_{12} – f_{15} . The overall performance of CCFR3 is better than the one of other CC algorithms on the CEC'2013 functions.

Fig. 5 shows the convergence of the CC algorithms. CCFR3 obtains the near global optimal solutions before 10^6 fitness evaluations on f_1 , f_4 and f_7 . CCFR3 converges faster than other CC algorithms on seven out of 11 fully and partially separable functions f_1 – f_{11} . This is because that the subpopulations in CCFR3 can have more chances to undergo evolution than the ones in other CC algorithms, which can improve

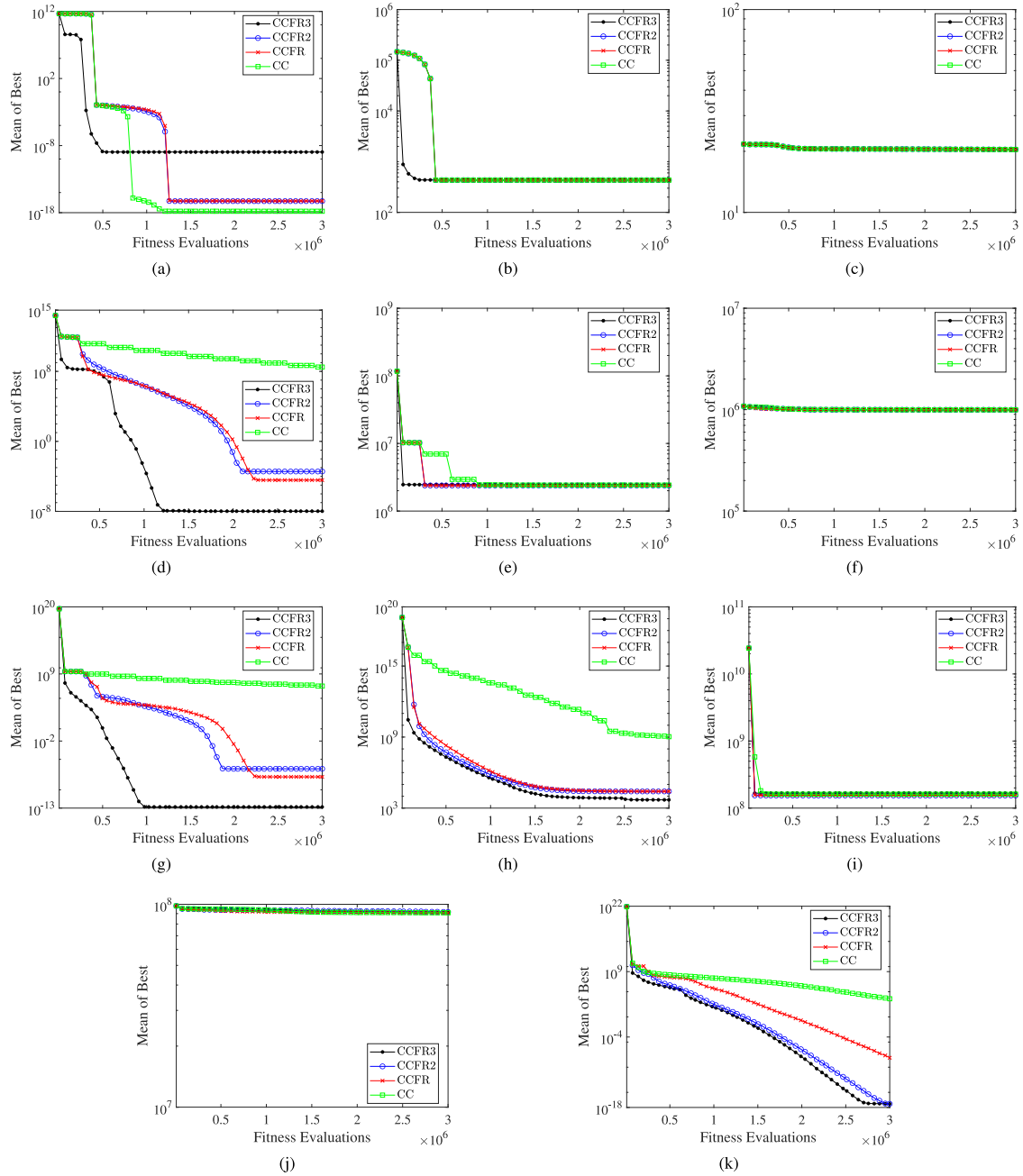


Fig. 5. Average convergence of algorithms over 25 independent runs on the CEC'2013 fully and partially separable functions. (a) f_1 . (b) f_2 . (c) f_3 . (d) f_4 . (e) f_5 . (f) f_6 . (g) f_7 . (h) f_8 . (i) f_9 . (j) f_{10} . (k) f_{11} .

the probability of obtaining the global optimal solution under a fixed computational budget.

GEs is a prespecified parameter of CCFR2 which specifies the number of evolutionary generations in one co-evolutionary cycle. CCFR3 is the same with CCFR2 except for that CCFR3 adopts self-adaptive contribution computation which removes GEs and adaptively determines the number of evolutionary generations (see Section 3). Table 4 shows the comparison between CCFR3 and CCFR2 with different values of GEs . There does not exist a suitable value of GEs for CCFR2 to solve each of the CEC'2013 functions at the same time. Although in the case of CCFR2 with $GEs = 1$ the subpopulations can have more chances to undergo evolution, the overall performance of CCFR2 with $GEs = 1$ is inferior to the one of CCFR3. This is because CCFR2 with $GEs = 1$ evaluates the contributions of subpopulations according to the evolutionary information in only one co-evolutionary cycle,

which results in that the contribution evaluation may be inaccurate. CCFR3 evaluates the contributions of the subpopulations when the subpopulations make stable contributions [see Eq. (3)]. There are no functions that CCFR3 performs significantly worse than its competitors. The overall performance of CCFR3 is superior to the one of CCFR2 with different values of GEs .

CCFR3 is also compared with MMO-CC (Peng et al., 2019), SHADE-ILS (Molina, LaTorre, & Herrera, 2018), MOS-CEC2013 (LaTorre, Muelas, & Pena, 2013) and MA-SW-Chains (Molina, Lozano, & Herrera, 2010). MMO-CC is a multimodal optimization enhanced CC algorithm. SHADE-ILS, MOS-CEC2013 and MA-SW-Chains were ranked the first in the IEEE CEC'2018, CEC'2013 and CEC'2010 competitions on large-scale global optimization, respectively. The parameters of the competitors of CCFR3 were set as in their publications. Table 5 summarizes the results of these algorithms. CCFR3 significantly outperforms its

Table 5

Average fitness values $\pm$ standard deviations over 25 independent runs on the CEC'2013 functions. The significantly best results are in bold font (Wilcoxon test with Holm p -value correction, $\alpha = 0.05$). b , n , and l have similar meanings as in Table 3.

F	CCFR3	MMO-CC	SHADE-ILS	MOS-CEC2013	MA-SW-Chains
f_1	1.10e-09 $\pm$ 2.40e-09	4.83e-20 $\pm$ 9.45e-21↓	2.69e-24 $\pm$ 1.35e-23↓	1.27e-22 $\pm$ 7.56e-23↓	8.49e-13 $\pm$ 1.11e-12
f_2	4.37e+02 $\pm$ 3.12e+01	1.53e+03 $\pm$ 7.42e+01↑	1.00e+03 $\pm$ 8.90e+01↑	8.32e+02 $\pm$ 4.57e+01↑	1.22e+03 $\pm$ 1.16e+02↑
f_3	2.04e+01 $\pm$ 4.80e-02	2.01e+01 $\pm$ 1.31e-02↓	2.01e+01 $\pm$ 1.12e-02↓	9.18e-13 $\pm$ 5.23e-14↓	2.14e+01 $\pm$ 5.73e-02↑
f_4	1.04e-08 $\pm$ 2.76e-08	2.96e+11 $\pm$ 3.70e+11↑	1.48e+08 $\pm$ 8.72e+07↑	1.74e+08 $\pm$ 8.03e+07↑	4.58e+09 $\pm$ 2.51e+09↑
f_5	2.46e+06 $\pm$ 4.89e+05	2.80e+06 $\pm$ 1.70e+06	1.39e+06 $\pm$ 2.03e+05↓	6.94e+06 $\pm$ 9.03e+05↑	1.87e+06 $\pm$ 3.13e+05↓
f_6	9.95e+05 $\pm$ 1.73e+03	1.06e+06 $\pm$ 3.21e+03↑	1.02e+06 $\pm$ 1.19e+04↑	1.48e+05 $\pm$ 6.56e+04↓	1.01e+06 $\pm$ 1.56e+04↑
f_7	1.55e-13 $\pm$ 7.25e-13	1.44e+10 $\pm$ 1.27e+10↑	7.41e+01 $\pm$ 5.46e+01↑	1.62e+04 $\pm$ 9.29e+03↑	3.45e+06 $\pm$ 1.29e+06↑
f_8	4.98e+03 $\pm$ 1.27e+03	1.77e+14 $\pm$ 1.34e+14↑	3.17e+11 $\pm$ 3.06e+11↑	8.00e+12 $\pm$ 3.14e+12↑	4.85e+13 $\pm$ 1.04e+13↑
f_9	1.67e+08 $\pm$ 2.98e+07	1.66e+08 $\pm$ 2.94e+07	1.64e+08 $\pm$ 1.57e+07	3.83e+08 $\pm$ 6.42e+07↑	1.07e+08 $\pm$ 1.71e+07↓
f_{10}	9.12e+07 $\pm$ 1.31e+06	9.39e+07 $\pm$ 7.14e+05↑	9.18e+07 $\pm$ 6.93e+05↑	9.02e+05 $\pm$ 5.17e+05↓	9.18e+07 $\pm$ 1.08e+06↑
f_{11}	5.25e-18 $\pm$ 3.81e-18	2.72e+12 $\pm$ 2.47e+12↑	5.11e+05 $\pm$ 2.25e+05↑	5.22e+07 $\pm$ 2.10e+07↑	2.19e+08 $\pm$ 3.04e+07↑
f_{12}	9.90e+02 $\pm$ 6.18e+01	8.98e+10 $\pm$ 2.60e+11↑	6.18e+01 $\pm$ 1.04e+02↓	2.47e+02 $\pm$ 2.59e+02↓	1.25e+03 $\pm$ 1.07e+02↑
f_{13}	4.73e+05 $\pm$ 7.44e+04	1.76e+12 $\pm$ 1.44e+12↑	1.00e+05 $\pm$ 7.19e+04↓	3.40e+06 $\pm$ 1.08e+06↑	1.98e+07 $\pm$ 1.86e+06↑
f_{14}	2.62e+07 $\pm$ 2.03e+06	3.54e+11 $\pm$ 4.77e+11↑	5.76e+06 $\pm$ 3.76e+05↓	2.56e+07 $\pm$ 8.11e+06	1.36e+08 $\pm$ 2.15e+07↑
f_{15}	2.16e+06 $\pm$ 2.13e+05	4.31e+08 $\pm$ 2.06e+08↑	6.25e+05 $\pm$ 2.40e+05↓	2.35e+06 $\pm$ 1.98e+05↑	5.71e+06 $\pm$ 7.73e+05↑
$b/n/l$	–	11/2/2	7/1/7	9/1/5	12/1/2

The symbols $\uparrow$ and $\downarrow$ have similar meanings as in Table 3.

competitors except SHADE-ILS on most of the CEC'2013 functions. CCFR3 performs significantly better than SHADE-ILS on seven functions, while performs significantly worse on seven functions. For fully separable functions f_1 – f_3 and fully nonseparable functions f_{12} – f_{15} , the performance of the algorithms does not differ much. CCFR3 performs significantly better than its competitors on f_4 , f_7 , f_8 and f_{11} by several orders of magnitude. CCFR3 is a competitive solver for the CEC'2013 problems.

5. Conclusion

In this paper, we presented a new CC framework named CCFR3 for large-scale global optimization. The parameter in CC algorithms (i.e., GEs), which specifies the number of evolutionary generations in each co-evolutionary cycle, determines when to compute the contribution of a subpopulation and select a subpopulation to undergo evolution. The value of GEs affects the performance of contribution-based CC algorithms. CCFR3 adopts a self-adaptive contribution computation which adaptively determines the number of evolutionary generations in each co-evolutionary cycle according to the evolutionary information, which means that there is no need to set the value of the parameter GEs in CCFR3.

CCFR3 was tested on the IEEE CEC'2013 large-scale benchmark functions. From the experimental results in this paper, several conclusions can be drawn. Firstly, CCFR3 usually sets a smaller number of evolutionary generations in each co-evolutionary cycle than the prespecified value of GEs in the existing CC algorithms. The adaptive number of evolutionary generations improves the performance of CCFR2. Secondly, CCFR3 can allocate computational resources among subpopulations more frequently than other contribution-based CC algorithms. The subpopulations in CCFR3 can have more chances to undergo evolution, which can improve the probability of obtaining the global optimal solution. The convergence of CCFR3 is faster than its competitors. Thirdly, CCFR3 with a grouping method (i.e., ERDG) for decomposing an optimization problem significantly outperforms its competitors on the CEC'2013 large-scale optimization problems, especially on partially separable problems.

CCFR3 is competitive in solving the optimization problems which can be decomposed into several subproblems. In the future, we plan to employ our proposed algorithm to solve some real-world optimization problems which involve a large number of decision variables, e.g., the shape-design problems which need many variables to represent a complex shape.

CRedit authorship contribution statement

Ming Yang: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft, Supervision, Writing – review & editing. **Aimin Zhou:** Resources, Formal analysis, Writing – original draft. **Xiaofen Lu:** Formal analysis, Writing – original draft. **Zhihua Cai:** Supervision. **Changhe Li:** Writing – review & editing. **Jing Guan:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was supported by the Open Research Project of the Hubei Key Laboratory of Intelligent Geo-Information Processing, China [grant number KLIGIP-2021B04]; the Guangdong Provincial Key Laboratory, China [grant number 2020B121201001]; and the Natural Science Foundation of Hubei Province, China [grant number 2019CFB584].

Software implementation

The MATLAB source code of the CCFR3 algorithm can be accessed from the following link: <https://github.com/ymzhongzhong/CCFR3>.

References

- Blanchard, J., Beauthier, C., & Carletti, T. (2019). A surrogate-assisted cooperative co-evolutionary algorithm using recursive differential grouping as decomposition strategy. In *2019 IEEE congress on evolutionary computation (CEC)* (pp. 689–696).
- Bojan-Dragos, C.-A., Precup, R.-E., Preitl, S., Roman, R.-C., Hedrea, E.-L., & Szedlak-Stinean, A.-I. (2021). GWO-based optimal tuning of type-1 and type-2 fuzzy controllers for electromagnetic actuated clutch systems. *IFAC-PapersOnLine*, 54(4), 189–194, 4th IFAC Conference on Embedded Systems, Computational Intelligence and Telematics in Control CESCIT 2021.
- Hansen, N. (2005). The CMA evolution strategy: A tutorial. Retrieved from <https://hal.inria.fr/hal-01297037> (ArXiv e-prints, arXiv:1604.00772, 2016).
- Hu, X., He, F., Chen, W., & Zhang, J. (2017). Cooperation coevolution with fast interdependency identification for large scale optimization. *Information Sciences*, 381, 142–160.
- LaTorre, A., Muelas, S., & Pena, J.-M. (2013). Large scale global optimization: Experimental results with MOS-based hybrid algorithms. In *IEEE congress on evolutionary computation* (pp. 2742–2749).

- Li, X., Tang, K., Omidvar, M. N., Yang, Z., & Qin, K. (2013). Benchmark functions for the CEC'2013 special session and competition on large scale global optimization. Retrieved from http://www.al-roomi.org/multimedia/CEC_Database/CEC2013/LargeScaleGlobalOptimization/CEC2013_LargeScaleGO_TechnicalReport.pdf.
- Li, X., & Yao, X. (2009). Tackling high dimensional nonseparable optimization problems by cooperatively coevolving particle swarms. In *IEEE congress on evolutionary computation* (pp. 1546–1553).
- Li, X., & Yao, X. (2012). Cooperatively coevolving particle swarms for large scale optimization. *IEEE Transactions on Evolutionary Computation*, 16(2), 210–224.
- Mei, Y., Omidvar, M., Li, X., & Yao, X. (2016). A competitive divide-and-conquer algorithm for unconstrained large-scale black-box optimization. *ACM Transactions on Mathematical Software*, 42(2), 13:1–13:24.
- Molina, D., LaTorre, A., & Herrera, F. (2018). SHADE with iterative local search for large-scale global optimization. In *2018 IEEE congress on evolutionary computation (CEC)* (pp. 1–8).
- Molina, D., Lozano, M., & Herrera, F. (2010). MA-SW-Chains: Memetic algorithm based on local search chains for large scale continuous global optimization. In *IEEE congress on evolutionary computation* (pp. 1–8).
- Omidvar, M. N., Kazimipour, B., Li, X., & Yao, X. (2016). CBCC3—a contribution-based cooperative co-evolutionary algorithm with improved exploration/exploitation balance. In *2016 IEEE congress on evolutionary computation (CEC)* (pp. 3541–3548).
- Omidvar, M. N., Li, X., Mei, Y., & Yao, X. (2014). Cooperative co-evolution with differential grouping for large scale optimization. *IEEE Transactions on Evolutionary Computation*, 18(3), 378–393.
- Omidvar, M., Li, X., Yang, Z., & Yao, X. (2010). Cooperative co-evolution for large scale optimization through more frequent random grouping. In *IEEE congress on evolutionary computation* (pp. 1–8).
- Omidvar, M. N., Li, X., & Yao, X. (2011). Smart use of computational resources based on contribution for cooperative co-evolutionary algorithms. In *Genetic and evolutionary computation conference* (pp. 1115–1122).
- Omidvar, M. N., Yang, M., Mei, Y., Li, X., & Yao, X. (2017). DG2: A faster and more accurate differential grouping for large-scale black-box optimization. *IEEE Transactions on Evolutionary Computation*, 21(6), 929–942.
- Peng, X., Jin, Y., & Wang, H. (2019). Multimodal optimization enhanced cooperative coevolution for large-scale optimization. *IEEE Transactions on Cybernetics*, 49(9), 3507–3520.
- Peng, X., Liu, K., & Jin, Y. (2016). A dynamic optimization approach to the design of cooperative co-evolutionary algorithms. *Knowledge-Based Systems*, 109, 174–186.
- Potter, M. A., & Jong, K. A. D. (1994). A cooperative coevolutionary approach to function optimization. In *Parallel problem solving from nature* (pp. 249–257). Heidelberg, Germany: Springer.
- Precup, R. E., David, R. C., Roman, R. C., Szedlak-Stinean, A. I., & Petriu, E. M. (2021). Optimal tuning of interval type-2 fuzzy controllers for nonlinear servo systems using slime mould algorithm. *International Journal of Systems Science*, (2), 1–16.
- Precup, R.-E., Hedrea, E.-L., Roman, R.-C., Petriu, E. M., Szedlak-Stinean, A.-I., & Bojan-Dragos, C.-A. (2021). Experiment-based approach to teach optimization techniques. *IEEE Transactions on Education*, 64(2), 88–94.
- Rigatos, G., Siano, P., Selisteanu, D., & Precup, R. E. (2017). Nonlinear optimal control of oxygen and carbon dioxide levels in blood. *Intelligent Industrial Systems*, 3(2), 61–75.
- Sarker, R., Mohammadian, M., & Yao, X. (Eds.). (2002). *Evolutionary optimization*. New York, NY, USA: Springer.
- Shi, Y., Teng, H., & Li, Z. (2005). Cooperative co-evolutionary differential evolution for function optimization. In *Advances in natural computation* (pp. 1080–1088). Heidelberg, Germany: Springer.
- Sun, Y., Kirley, M., & Halgamuge, S. K. (2015). Extended differential grouping for large scale global optimization with direct and indirect variable interactions. In *GECCO '15, Proceedings of the 2015 annual conference on genetic and evolutionary computation* (pp. 313–320). New York, NY, USA: ACM.
- Sun, Y., Kirley, M., & Halgamuge, S. K. (2018). A recursive decomposition method for large scale continuous optimization. *IEEE Transactions on Evolutionary Computation*, 22(5), 647–661.
- Sun, Y., Omidvar, M. N., Kirley, M., & Li, X. (2018). Adaptive threshold parameter estimation with recursive differential grouping for problem decomposition. In *GECCO '18, Proceedings of the genetic and evolutionary computation conference* (pp. 889–896). New York, NY, USA: ACM.
- Tan, G. W.-H., Ooi, K.-B., Leong, L.-Y., & Lin, B. (2014). Predicting the drivers of behavioral intention to use mobile learning: A hybrid SEM-neural networks approach. *Computers in Human Behavior*, 36, 198–213.
- Tezuka, M., Munetomo, M., & Akama, K. (2004). Linkage identification by nonlinearity check for real-coded genetic algorithms. In *Conference on genetic and evolutionary computation* (pp. 222–233). ACM.
- Van den Bergh, F., & Engelbrecht, A. P. (2004). A cooperative approach to particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, 8(3), 225–239.
- Weise, T., Chiong, R., & Tang, K. (2012). Evolutionary optimization: Pitfalls and booby traps. *Journal of Computer Science and Technology*, 27(5), 907–936.
- Yang, M., Li, C., Cai, Z., & Guan, J. (2015). Differential evolution with auto-enhanced population diversity. *IEEE Transactions on Cybernetics*, 45(2), 302–315.
- Yang, M., Omidvar, M. N., Li, C., Li, X., Cai, Z., Kazimipour, B., et al. (2017). Efficient resource allocation in cooperative co-evolution for large-scale global optimization. *IEEE Transactions on Evolutionary Computation*, 21(4), 493–505.
- Yang, Z., Tang, K., & Yao, X. (2008a). Large scale evolutionary optimization using cooperative coevolution. *Information Sciences*, 178(15), 2985–2999.
- Yang, Z., Tang, K., & Yao, X. (2008b). Multilevel cooperative coevolution for large scale optimization. In *IEEE congress on evolutionary computation* (pp. 1663–1670).
- Yang, P., Tang, K., & Yao, X. (2019). A parallel divide-and-conquer-based evolutionary algorithm for large-scale optimization. *IEEE Access*, 7, 163105–163118.
- Yang, M., Zhou, A., Li, C., Guan, J., & Yan, X. (2020). CCFR2: A more efficient cooperative co-evolutionary framework for large-scale global optimization. *Information Sciences*, 512, 64–79.
- Yang, M., Zhou, A., Li, C., & Yao, X. (2021). An efficient recursive differential grouping for large-scale continuous problems. *IEEE Transactions on Evolutionary Computation*, 25(1), 159–171.